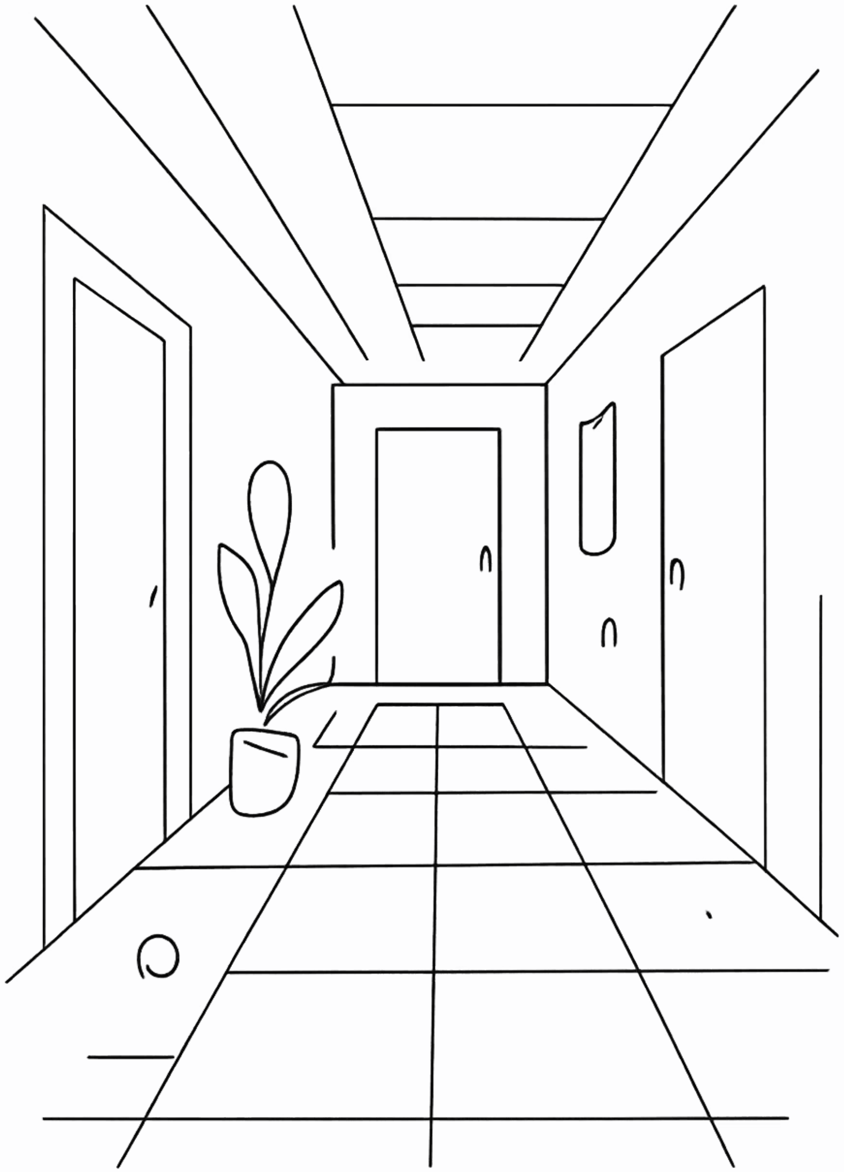


SURA Indoor Positioning — Modeling Report

The core challenge is to learn the environment fingerprint—not the specific walked trajectory—while meeting the deployment constraint of strictly causal, real-time, on-device inference with no look-ahead.

The model must learn a static, spatially unique, temporally stable fingerprint, rather than memorizing walked trajectories. This requires separating spatial features from temporal motion features so the network captures location identity instead of path-specific dynamics.

All components must be strictly causal with no look-ahead and no bidirectional processing. This rules out bidirectional RNNs.

The model must generalize to unseen devices and unseen sessions. Evaluation should prefer per-node ground truth from the static fingerprint database over interpolated continuous-walk labels, so performance reflects true node-level localization accuracy.

Prior models' architecture review

Model 1 — Multi-branch CNN + Bi-LSTM

Implemented in `train_cnn_lstm.py`. This model used two feature branches: a WiFi branch with `Conv1d(64)` filters and an IMU branch with `Conv1d(32)` filters. Their outputs were concatenated and passed into a `Bi-LSTM(128 hidden, bidirectional=True)`, followed by fully connected layers that predicted a single absolute (X, Y) position for each 10-frame window. Trained with MSE loss on absolute coordinates. It had the best raw errors, but the bidirectional LSTM explicitly looked ahead inside the 10-frame window, making the model **non-causal and unsuitable for strict real-time deployment**.

Reported performance on S9+ was **5.04 m MAE** with a maximum error of **17.4 m**.

Model 2 — Causal Hybrid (CausalConv + uni-LSTM)

Implemented in `train_causal_hybrid.py`. The key change was replacing standard convolutions with `CausalConv1d`, which pads only on the left side so the **network never sees future frames**, and replacing the Bi-LSTM with a unidirectional LSTM (`bidirectional=False`). Instead of predicting absolute position directly, this model predicts frame-to-frame motion deltas (dx, dy), which are then cumulatively summed at test time starting from the known start position. This turns localization into a motion-estimation problem, avoiding look-ahead cheating.

It achieved **3.64 m MAE** on test.

Model 4 — Neural EKF (learned α-gate)

Implemented in `train_ekf_fusion.py`. This model used a three-stage fusion design. First, a spatial anchor branch combined WiFi and magnetic-field MLPs to produce a per-frame spatial estimate $P_{spatial}$. Second, a motion tracker built from `CausalConv` plus a unidirectional LSTM predicted motion deltas. Third, a learned Kalman-gain-like α gate blended the two signals dynamically through a sigmoid. The training objective was a composite loss with weights `1.0` on the final output, `0.2` on the spatial branch, and `2.0` on the motion branch, encouraging both stable absolute anchors and accurate motion tracking. At inference time, the fusion was auto-regressive: $P_{final}[t] = \alpha[t] \cdot P_{spatial}[t] + (1 - \alpha[t]) \cdot (P_{final}[t-1] + \delta[t])$.

This model reported **4.29 m MAE**, and the average α of about **0.6** suggests it slightly preferred the spatial anchor over motion estimate.

Why earlier results were misleading

- ## Single 1-D path Training data

  All four Continuous_Fused_*.csv files trace the same corridor, with the same start point (≈(90, 24)) and the same end region, and only **2 turns** along the way, only about **325 training windows** and **99 test windows** — all drawn from one narrow line through a 2-D building. The models never saw a true 2-D sampling of the environment; they only saw a single trajectory repeated across files. That is why they do not learn a general fingerprint of space.

- ## 2. Magnetometer was direction-dependent

  Raw Mag_x, Mag_y, and Mag_z are measured in the phone's **body frame**, so they rotate whenever the user changes heading. The same physical spot therefore produces different axis values when walked in opposite directions, even though the location is identical. It was learning a **direction-dependent** signature . Model 3 (Environment Model), achieved **6.36 m MAE** on the forward path and collapsed to **63 m MAE** on the reversed path. The model had effectively learned a heading-conditioned shortcut, so when the travel direction flipped, the supposed fingerprint no longer matched.

- ## 3. Single (X,Y) regression blurs multi-modality

  A single WiFi/magnetic fingerprint measurement can correspond to multiple points along the corridor because several places look similar in sensor space. But a regression model forced to emit only one (X, Y) cannot represent that ambiguity. Instead of saying "this reading matches locations **A**, **B**, and **C** with comparable likelihood," the network must collapse everything into one coordinate, effectively predicting the **centroid** of the candidate set. That averaging blurs the true multi-modal structure of the problem and systematically pulls predictions toward the route center, where the competing hypotheses overlap most. So even when the signal is informative, the representation is wrong: the model is not estimating a distribution over possible locations, only a single point estimate. This is a fundamental limitation of the target formulation, not just a tuning issue.

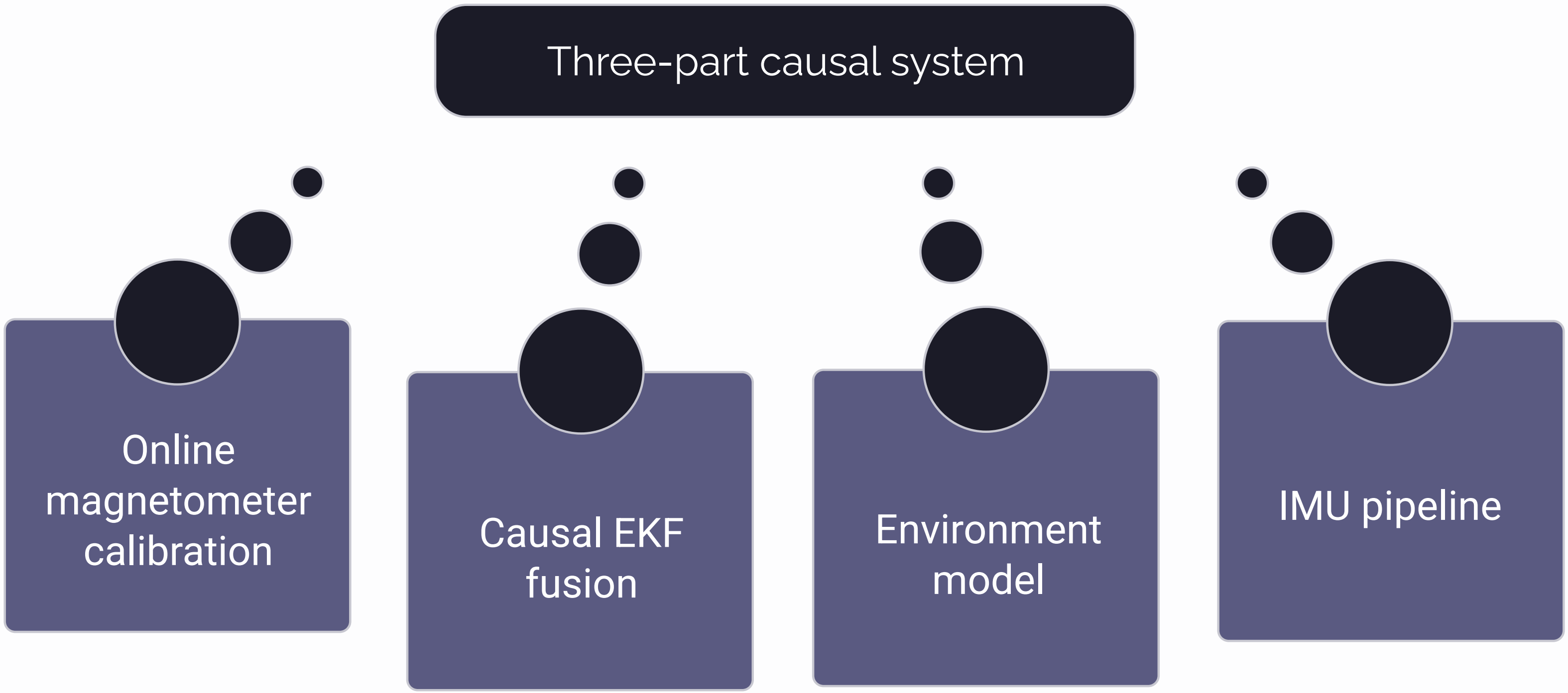
- ## 4. Fabricated ground truth

  The Continuous_Fused_*.csv files were generated by fuse_continuous_wifi.py, which used **BE Building** coordinates rather than the IT Engineering layout, linearly interpolated True_X/True_Y by timestamp from the ordered static-node list, and simulated WiFi measurements by injecting Gaussian noise around the static node scans. That means the "ground truth" was not independently observed at all — it was constructed from the **ordering of static nodes mapped onto time**. In practice, the models were being scored against an invented path that had been synthesized from the same database used to generate the inputs. This is exactly why route memorization looked successful: the labels themselves formed a sequence, so a model that learned the sequence could appear accurate without ever solving real localization. The reported errors therefore reflected consistency with the fabricated trajectory, not validity against real-world ground truth.

New architecture - learned environment + causal fusion

We separate the problem into components that can be trained and validated independently on the static fingerprint database, instead of learning from continuous walks. This fundamental shift enforces the environment model to learn from dense static scans with **538 node visits** covering **167/168 nodes**, and the fusion stage remains causal and nearly parameter-free.

Static fingerprint learning + causal motion fusion + online calibration



Self-calibration inside the EKF using a trailing buffer of raw samples. Fits an ellipsoid. Reduces cross-device spread by 3×.

Classical Kalman fusion of the heatmap measurement and PDR prediction. PDR is smooth but drifting.

Per-frame WiFi heatmap output. No temporal processing; outputs a probability distribution over grid cells rather than a single point.

Causal PDR motion predictions. Step detection from accelerometer magnitude.

Key properties

- Environment trained exclusively on a static fingerprint database. It is never trained on continuous walks, so it learns a location fingerprint rather than a trajectory.
- Instead of predicting a single *(X, Y)*, the environment model outputs a probability distribution over a **1 m grid**. The spatial spread of that heatmap becomes the Kalman measurement covariance **R**, so ambiguous measurements are automatically down-weighted in fusion.
- The **OnlineMagNormalizer** fits an ellipsoid from a trailing buffer of raw samples. This reduces cross-device **||M||** spread by **3×** (**1.77 → 0.57 μT**), enabling the same model to work across different phones without per-device calibration.
- The EKF is a standard closed-form filter with no learned parameters beyond the environment model. It cannot memorize a route because it has no memory of the path sequence—only the current state plus the measurement and motion models—so the system remains fully causal.

Reported results

WiFi heatmap environment model: **MAE 1.43 m** on the random split and **2.02 m** on the held-out device (S9+), a **2.5× improvement** over the best earlier model (**3.64 m**). The random split evaluates generalization to unseen visits of the same nodes, while the held-out-device split tests generalization to an unseen phone; both are strong results.

The online magnetometer normalizer reduced cross-device **||M||** spread by about **3×**. This was validated by running the normalizer on all four continuous walks and measuring the standard deviation of **||M||** across devices before and after calibration.

Causal PDR achieves about **1.5 m** short-walk MAE. That drift on long walks is expected and unavoidable without the WiFi anchor: PDR is the smooth, causal motion model, while the WiFi heatmap provides the drift-free correction every **~1 second**.

3.64m to 1.43m - Evolution of Positioning Models

Initial Models (3.64–6.36 m MAE)

- Model 1 (CNN-LSTM): 5.04 m — Non-causal, bidirectional look-ahead
- Model 2 (Causal Hybrid): 3.64 m — Causal but memorizes route
- Model 3 (Environment): 6.36 m → 63 m reversed — Direction-dependent features
- Model 4 (Neural EKF): 4.29 m — Learned fusion, still memorizes

Current (1.43 m MAE)

- Model 5 (WiFi Heatmap): 1.43 m random, 2.02 m held-out device
- 2.5× improvement over best prior model
- Trains on real static data (538 node visits)
- Outputs probability heatmap (not single point)
- Trivially direction-invariant (per-frame processing)
- Cannot memorize route (no trajectory training)

Metric	Initial Models	Current
MAE progression	6.36 → 5.04 → 4.29 → 3.64 m	**1.43 m**
Route memorization risk	High → High → High → Medium	**None**
Training data	Continuous walks	**Static DB**
Output	Single point	**Heatmap**

The Path Forward

> The WiFi heatmap (1.43 m) is proven. The full EKF fusion is ready. The only missing piece is WiFi-enabled walks with per-frame ground truth. Once that data is collected, the complete system will be validated end-to-end and ready for deployment.